

NDNc – a lightweight integration of ndn-cxx with NDN-DPDK to achieve high throughput performance in scientific applications

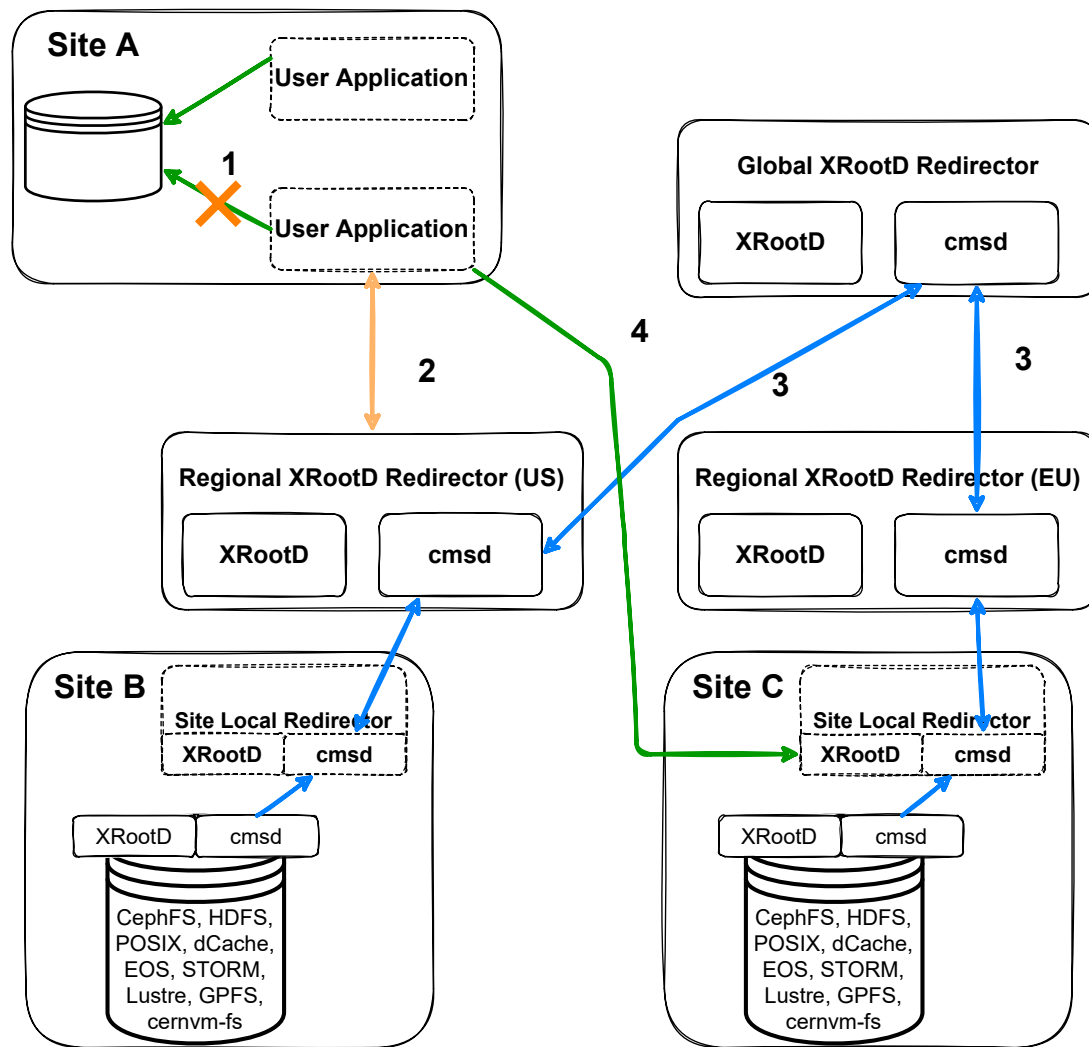
Cătălin Iordache

Introduction

The goal of the SANDIE and the N-DISE projects was and is to deploy Named Data Networking in Large-Scale Data-Intensive Science experiments such as LHC high energy physics

- The way data is being accessed and delivered today at the CMS experiment
- How a deployed NDN application will look like at CMS
- The throughput performance
- NDNc and plans for the library

How are CMS jobs accessing data today?



Multiple CMSSW Jobs are running at one site. Initially will try to open the file locally:

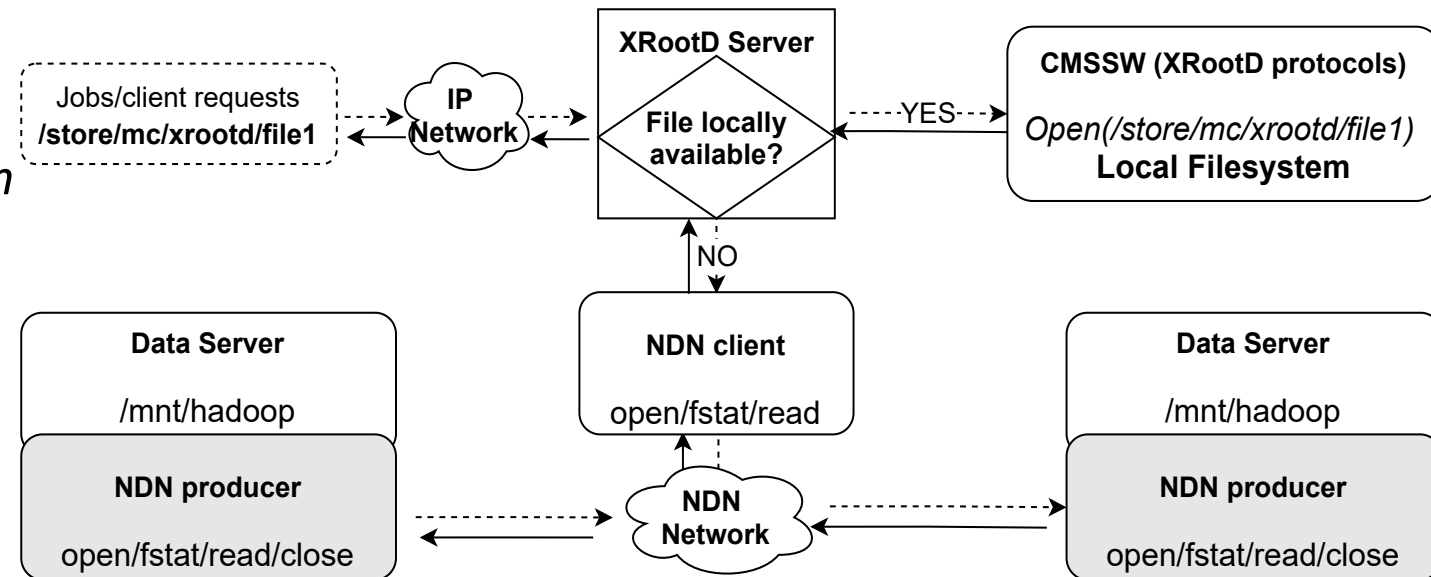
- if **FILE FOUND** then start reading data
- if **FILE NOT FOUND** then go to the next step
- ask the **Regional Redirector** to find the location of the file needed. The Regional Redirector will go as far as asking the **Global Redirector** for the file location.
 - if the file exists, it will return an IP address for XRootD to connect to

XRootD and the Open Storage System plugin

- XRootD is a framework
 - the file management tool used at CERN
 - enables developers to implement various types of plug-ins (e.g., filesystem, caching, security)
 - written in C++ but there are also on-going version for a Golang one
- **Open Storage System plugin** (i.e., storage system implementation)
 - it's a C++ dynamic library that exports *XrdOssGetStorageSystem* symbol
 - specified in an XRootD configuration file and loaded at run time, it accepts parameters (the plugin can be configurable)
 - offers support for all related POSIX file system calls (e.g., open, close, read, opendir, readdir, ...)

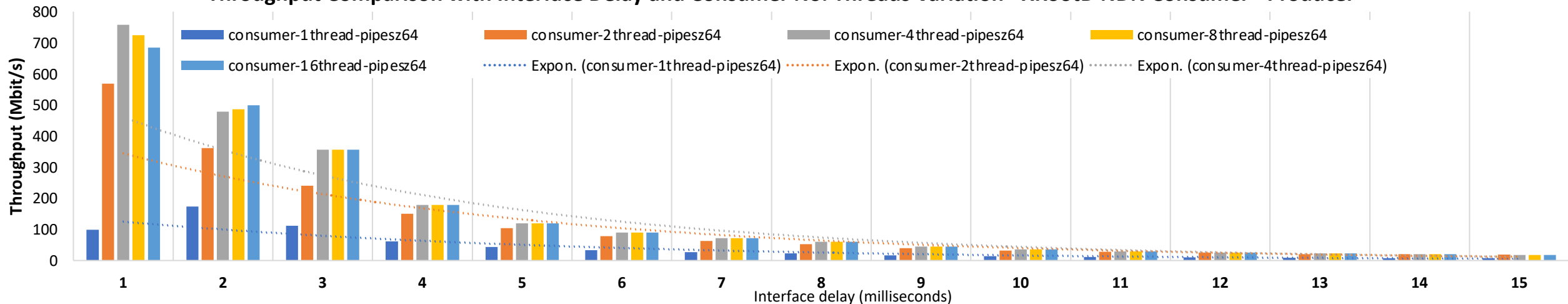
NDN based XRootD OSS plugin

- Embed an NDN consumer into an XRootD plugin
- Translates each file system call into an NDN name
- **/ndn/xrootd/meta/store/mc/file1** for *open* and *fstat*
- **/ndn/xrootd/store/mc/file1/seg=128?v=1** for *read*
- Sends Interests over the NDN network
- Receives Data packets, validates them and replies to XRootD worker threads
- For the *read* requests, the producer will associate a segment number with an offset from a file to read depending the pre-defined payload size

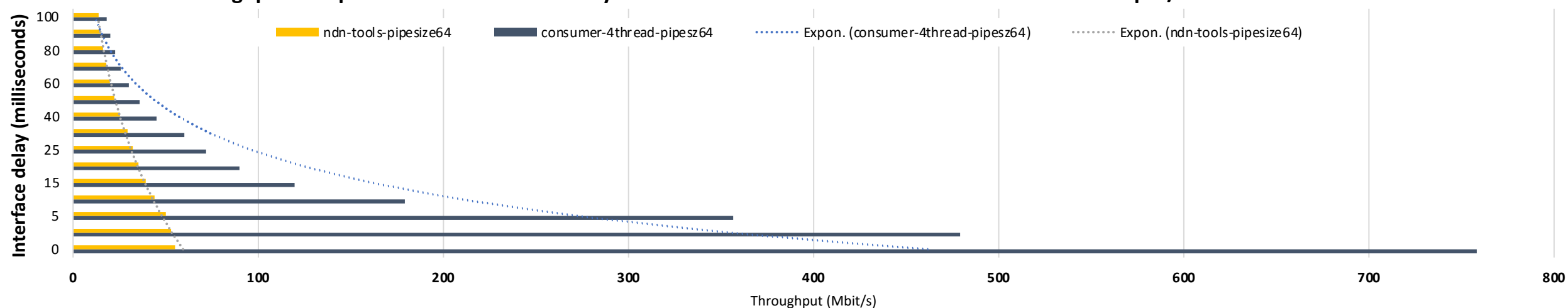


Comparing with other NDN applications

Throughput Comparison with Interface Delay and Consumer No. Threads Variation - XRootD NDN Consumer - Producer

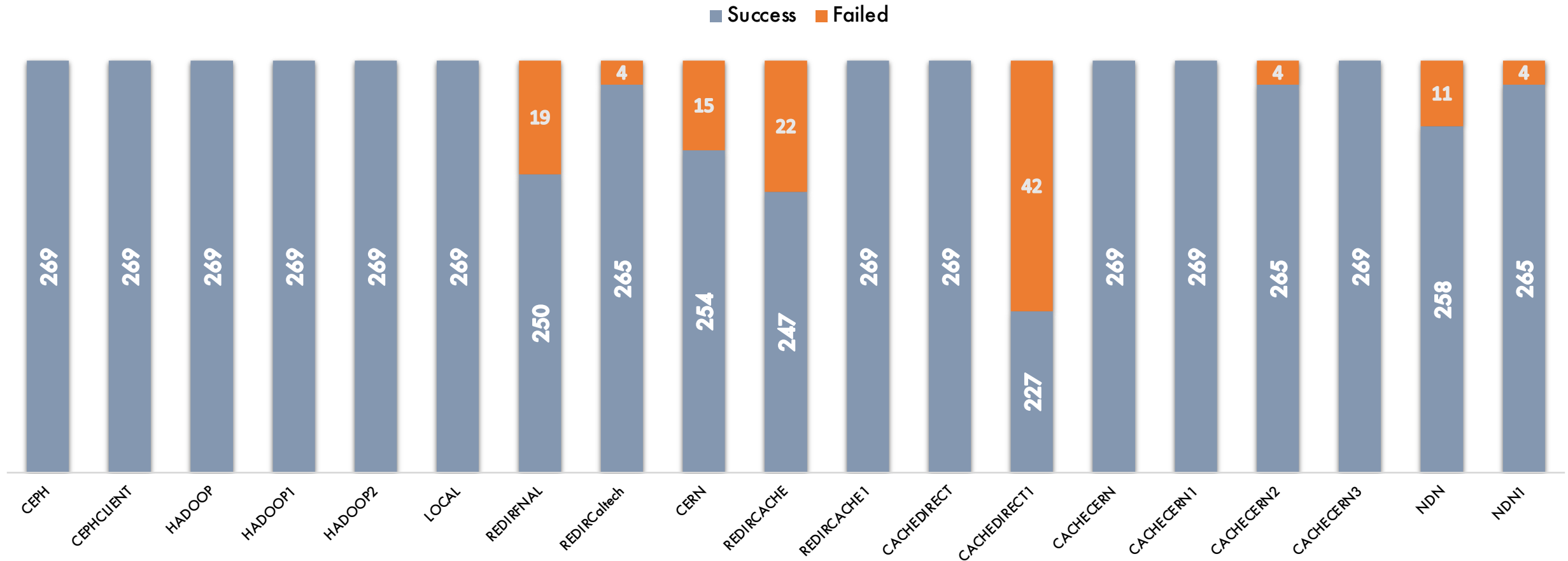


Throughput Comparison with Interface Delay Variation - XRootD NDN Consumer - Producer vs ndn put/cat chunks on-demand



Comparing with other storage solutions

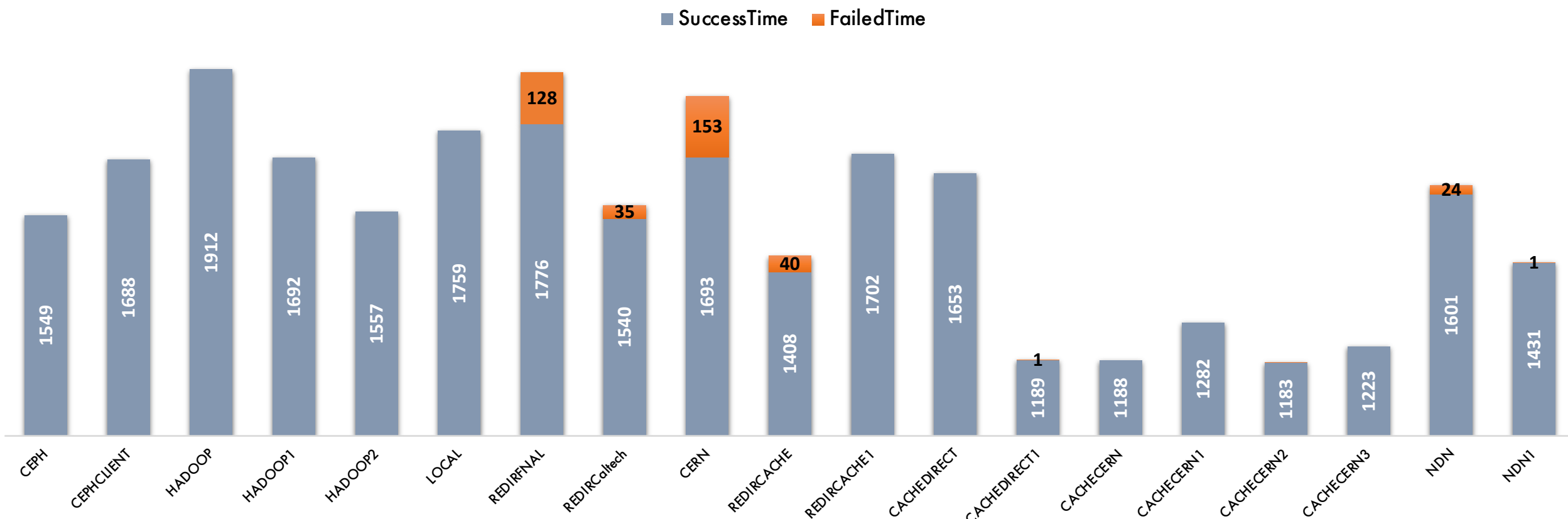
Success rate of 269 jobs running CMSSW on different storage solutions at CMS



Input Dataset: /DYJetsToLL_M-50_TuneCUETP8M1_13TeV-amcatnloFXFX-pythia8/RunIISummer16MiniAODv3-PUMoriond17_94X_mcRun2_asymptotic_v3_ext2-v1/MINIAODSIM

Comparing with other storage solutions

Total run time in hours of 269 jobs running CMSSW on different storage solutions at CMS

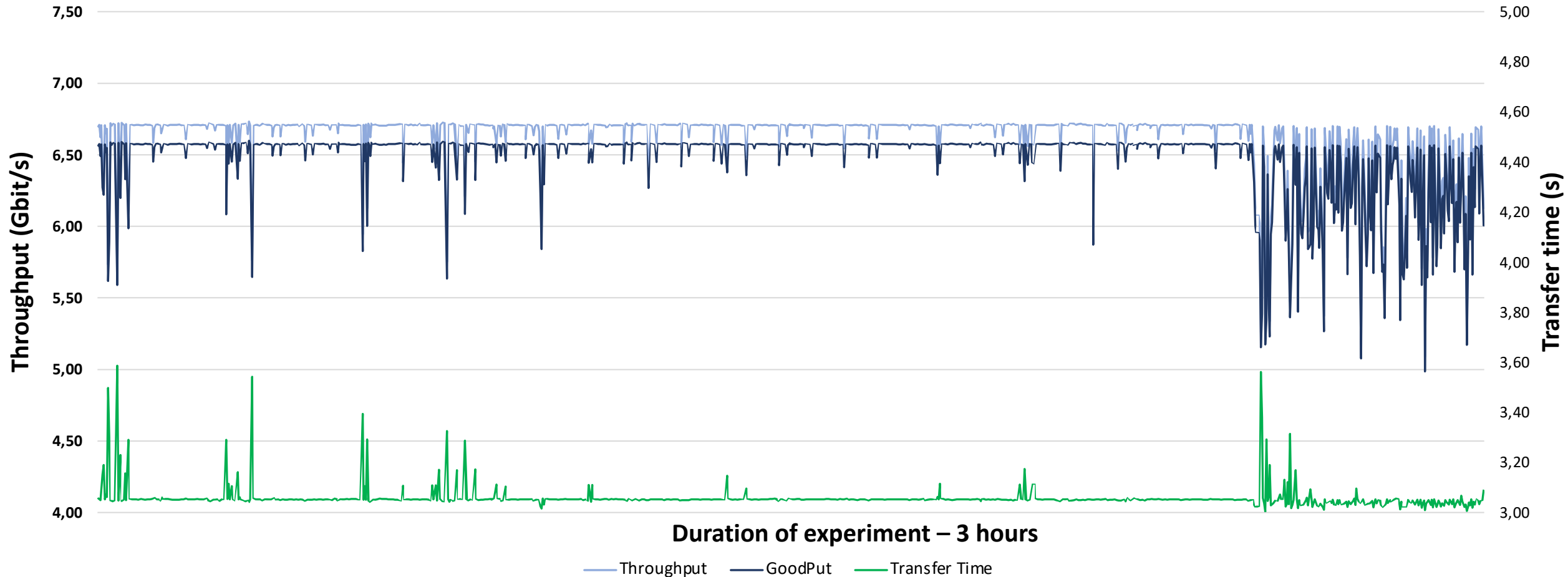


Input Dataset: /DYJetsToLL_M-50_TuneCUETP8M1_13TeV-amcatnloFXFX-pythia8/RunIISummer16MiniAODv3-PUMoriond17_94X_mcRun2_asymptotic_v3_ext2-v1/MINIAODSIM

In search for more performance

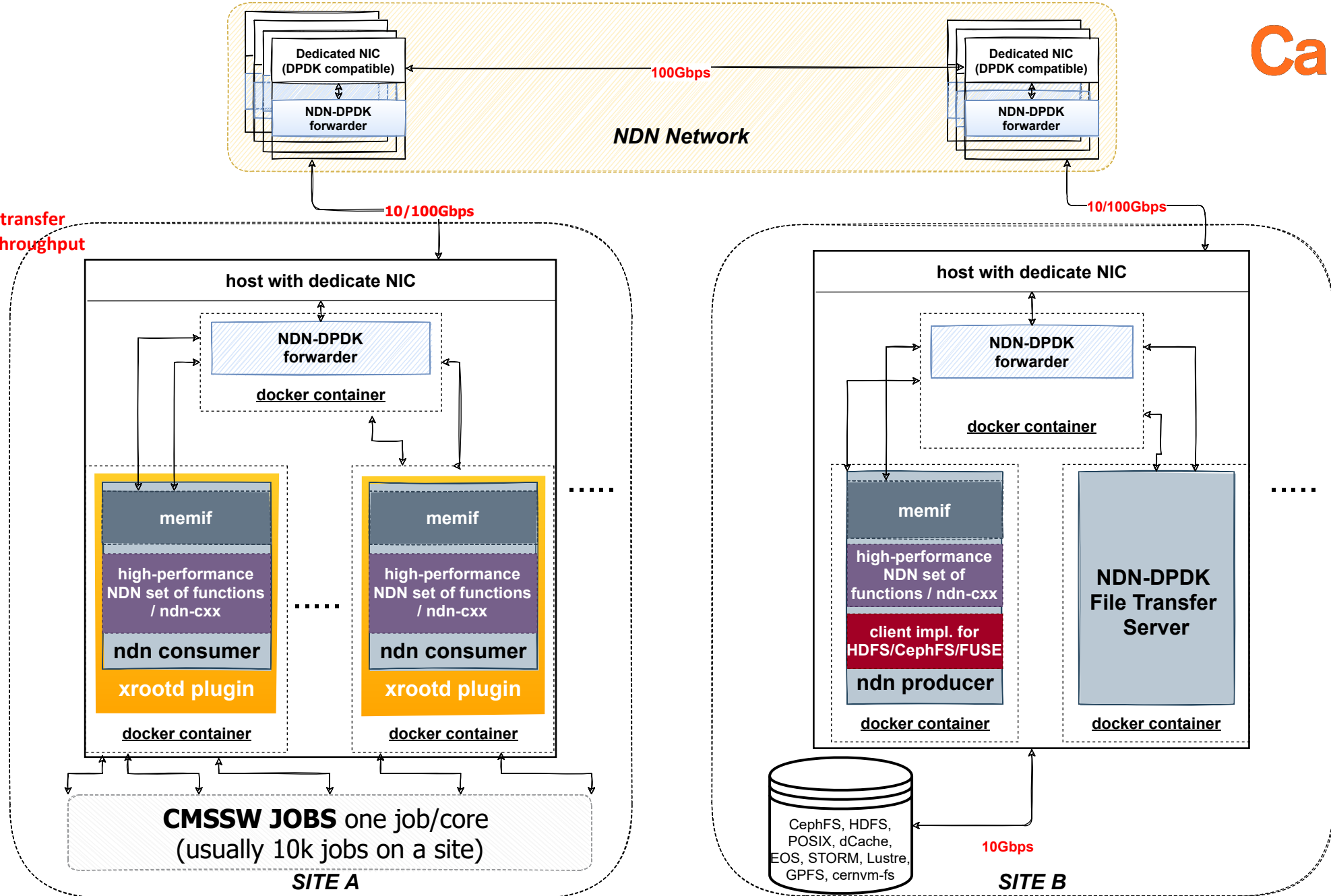
- NFD was the main bottleneck which limits applications to ~1Gbps
- Implemented NDN DPDK consumer-producer file transfer applications
 - demo at Super Computing 2019
 - good performance on single a thread: 6.7Gbps over WAN
 - very hard to setup (hardware and software requirements), especially on CentOS 7
 - no NDN API
- Implemented NDNgo consumer and producer file transfer applications
 - performance around 2.2Gbps per consumer
 - the API was never intended for high throughput performance, but for NDN-DPDK forwarder's management

NDN-DPDK Based Consumer-Producer Transfer File Application Throughput at SC19
Consumer (booth/Denver) - NDN-DPDK Forwarder (booth/Denver) - Producer (Caltech California)



Performance goals:

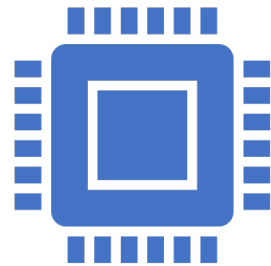
- ~10 Gbps for a single transfer
- 100 Gbps aggregate throughput



NDNc so far

- NDNc: ndn-cxx support to NDN-DPDK
 - <https://github.com/cmscaltech/sandie-ndn/tree/master/NDNc>
 - uses memif shared memory packet interface that provides performance packet transmit and receive between user application and VPP
- Already implemented:
 - GraphQL C++ client able to configure the local NDN-DPDK forwarder (*createFace*, *insertFibEntry* and *delete* JSON mutations available)
 - single threaded memif transport layer to transmit/receive bursts of packets at a time
 - face transmit/receive one or many *ndn::Blocks* in a single burst
 - PIT token support
 - uses ndn-cxx to encode/decode L2 and L3 packets
- Abstract PacketHandler class to base all consumer and producer applications
 - easier porting of existing code or integration of other applications
- Congestion window and retransmission: fixed and aimd (more to be added soon)
- Logger class
- CMake, Dockerfile

NDNc applications and preliminary results



PING consumer and producer

80-200 μ s RTT

with 6144 Bytes payload size



FILE TRANSFER consumer and producer

4.3 Gbps

with 6600 Bytes payload size

NDNc future plans

- Extensive benchmarking to understand the current behaviour and the maximum throughput performance that can be achieved
- Identify possible bottlenecks and fix them
- If necessary, replace key ndn-cxx functions such as the decoder in order to further boost the performance
- Add multi-threaded support to memif and pipelines
- Port the NDN XRootD plugin developed during the SANDIE project to NDNc
- Extend the number of consumer applications to HDFS/CephFS/Fuse

★ We wish more will join our efforts

References

- *Named Data Networking based File Access for XRootD*. 24th International Conference on Computing in High Energy and Nuclear Physics (CHEP 2019)
<https://doi.org/10.1051/epjconf/202024504018>
- *SANDIE NDN XRootD plugin using NFD forwarder*: <https://github.com/cmscaltech/sandie-ndn/tree/master/xrootd-ndn-oss-plugin>
- SANDIE DPDK consumer and producer applications used during at SC19 demo:
<https://github.com/cmscaltech/sandie-ndn/tree/master/xrootd-ndn-dpdk-oss-plugin>
- NDNgo file transfer consumer and producer applications: <https://github.com/cmscaltech/sandie-ndn/tree/master/go>
- NDNc: <https://github.com/cmscaltech/sandie-ndn/tree/master/NDNc>